Informe 1

Autores:

Eric Guacache C.I: 31.788.759
Renny Andrade C.I: 29.727.828

Índice

1. Implementación de la Recursividad y Uso de la Pila en MIPS32 **3**
1.1. Mapeo de Registros y Control de Flujo 3
1.2. Preservación del Estado en Funciones Recursivas 3
1.3. Mecanismo de Desapilado y Retorno 3

2. Riesgos de Desbordamiento y Estrategias de Mitigación **3**
2.1. Desbordamiento de Pila (Stack Overflow) 3
2.2. Desbordamiento Aritmético (Arithmetic Overflow) 4

3. Comparación entre Implementación Iterativa y Recursiva **4**

4. Comparación entre Ejemplos Académicos y Programas Operativos Complejos **4**

5. Justificación del Enfoque: Determinación de Paridad en MIPS32 **4**
5.1. Eficiencia de Memoria 5
5.2. Tiempo de Ejecución 5
5.3. Claridad y Mantenibilidad del Código 5

6. Análisis y Discusión de Resultados **5**
6.1. Evaluación de la Función Recursiva 5
6.2. Evaluación de la Función Iterativa 5
6.3. Conclusiones Comparativas 6

1. Implementación de la Recursividad y Uso de la Pila en MIPS32

Para implementar subrutinas recursivas en la arquitectura MIPS32, es fundamental la utilización coordinada de los registros argumentales (`$a0--$a3`), el puntero de pila (`$sp`) y el registro de dirección de retorno (`$ra`), junto con las instrucciones de salto con enlace (`jal`) y salto a registro (`jr`).

1.1. Mapeo de Registros y Control de Flujo

- **Registros de Argumentos (`$a0--$a3`):** Almacenan los parámetros de entrada requeridos por la función antes de realizar la invocación.
- **Instrucción `jal` (Jump and Link):** Realiza una bifurcación hacia la etiqueta asignada a la subrutina y, simultáneamente, guarda la dirección de la siguiente instrucción en el registro `$ra`.
- **Stack Pointer (`$sp`):** Es un registro especial que apunta al tope actual de la memoria pila en RAM. Dado que la pila crece hacia direcciones de memoria menores, la reserva de espacio requiere restar bytes a `$sp` en múltiplos de 4 (por ejemplo, `addi $sp, $sp, -8` para reservar espacio para 2 palabras de 32 bits).

1.2. Preservación del Estado en Funciones Recursivas

Debido a que cada llamada recursiva sobrescribe los valores almacenados en `$ra` y en los registros de argumentos `$a#`, es indispensable salvar dichos valores en el *stack frame* (marco de pila) antes de ejecutar un nuevo llamado recursivo.

Toda función recursiva debe definir de manera explícita un **caso base** mediante instrucciones condicionales (`beq`, `bne`) para detener el apilamiento y comenzar el retorno.

1.3. Mecanismo de Desapilado y Retorno

Para retornar al flujo de ejecución anterior:

1. Se restauran de la memoria pila los datos previamente respaldados utilizando la instrucción `lw` (Load Word), recuperando la dirección de retorno original en `$ra` y los argumentos en `$a#`.
2. Se libera el espacio del marco de pila sumando la cantidad equivalente de bytes al registro `$sp` (`addi $sp, $sp, 8`).
3. Se ejecuta la instrucción `jr $ra` (Jump Register) para bifurcar de regreso a la dirección guardada en `$ra`.

2. Riesgos de Desbordamiento y Estrategias de Mitigación

2.1. Desbordamiento de Pila (Stack Overflow)

Existe un límite finito de memoria RAM asignado a la pila del proceso. En cada llamada recursiva se asigna espacio adicional para guardar los registros locales. Si la recursión se extiende demasiado de manera descontrolada y `$sp` intenta acceder a regiones de memoria fuera del límite de la pila, el sistema operativo abortará la ejecución por una violación de segmento (*segmentation fault*).

Mitigación:

- **Validación de entrada:** Sanitizar y restringir los valores de entrada para garantizar que el número de iteraciones no exceda la capacidad física de la pila.
- **Definición rigurosa del caso base:** Garantizar que la condición de parada del algoritmo recursivo sea inalcanzable de manera infinita.

2.2. Desbordamiento Aritmético (Arithmetic Overflow)

Los registros generales de MIPS32 tienen un tamaño fijo de 32 bits, lo que permite almacenar enteros con signo dentro del rango [−2,147,483,648, 2,147,483,647]. Si una función recursiva genera valores que crecen de forma exponencial por iteración y supera dicho rango, la ALU de MIPS genera una excepción por desbordamiento aritmético.

Mitigación: Utilizar instrucciones aritméticas sin detección de desbordamiento, tales como `addu`, `addiu` o `subu`. Estas instrucciones ignoran la excepción de hardware y permiten continuar la ejecución, aunque es responsabilidad del programador gestionar el manejo de sobreflujo de datos.

3. Comparación entre Implementación Iterativa y Recursiva

- **Implementación Iterativa:** Opera principalmente utilizando los registros internos del procesador (registros temporales `$t0--$t9`). No requiere interacción continua con la memoria principal (RAM) para el control de bucles, manteniendo un consumo espacial de $\mathcal{O}(1)$.
- **Implementación Recursiva:** Depende fuertemente del acceso a la memoria RAM a través del puntero de pila `$sp` para conservar el estado de la función (`$ra` y `$a#`). Adicionalmente, añade una carga computacional considerable por el uso repetido de las instrucciones `jal` y `jr`.

4. Comparación entre Ejemplos Académicos y Programas Operativos Complejos

Los ejemplos presentados en la literatura académica suelen centrarse en algoritmos aislados de baja escala orientados a ilustrar sintaxis y conceptos básicos. En contraste, una aplicación operativa real en MIPS32 exige estructurar problemas complejos mediante la descomposición modular en múltiples subrutinas interconectadas, integrando manejo de memoria, operaciones de entrada/salida mediante `syscall` y cadenas de llamadas a subfunciones.

5. Justificación del Enfoque: Determinación de Paridad en MIPS32

Para determinar la paridad de un entero positivo n, la implementación iterativa resulta sustancialmente superior a la solución recursiva bajo tres criterios fundamentales:

5.1. Eficiencia de Memoria

- **Iterativo ($\mathcal{O}(1)$):** Opera exclusivamente dentro de los registros del procesador (`$t0, $t1, $t2`), con cero accesos a la memoria RAM.
- **Recursivo ($\mathcal{O}(n)$):** Requiere la asignación de n marcos de pila en la RAM, incrementando linealmente el uso de memoria y presentando riesgo de desbordamiento de pila para valores elevados de n.

5.2. Tiempo de Ejecución

Aunque ambos enfoques exhiben una complejidad temporal teórica de $\mathcal{O}(n)$, la solución iterativa ejecuta únicamente instrucciones simples de un solo ciclo (`add`, `subi`, `neg`, `beq`). Por su parte, la solución recursiva incurre en una alta penalización de ciclos por el consumo de transferencias en RAM (`sw`, `lw`), modificaciones de `$sp` y saltos de subrutina (`jal`, `jr`).

5.3. Claridad y Mantenibilidad del Código

El enfoque iterativo genera un flujo de control lineal, directo y fácil de depurar, a diferencia del enfoque recursivo cuyo rastreo de estado requiere seguir la evolución de la memoria pila.

6. Análisis y Discusión de Resultados

6.1. Evaluación de la Función Recursiva

- **Gestión de la Pila (`$sp`):** En cada llamada se reservan 8 bytes mediante `addi $sp, $sp, -8` para almacenar la dirección de retorno (`sw $ra, 4($sp)`) y el valor decrementado del parámetro (`sw $a0, 0($sp)`), produciendo una ocupación lineal $\mathcal{O}(n)$.
- **Instrucciones y Registros:** Utiliza los registros `$sp`, `$ra`, `$a0`, `$v0` (retorno de resultado), `$zero` y `$t0`. Aplica las instrucciones `sw`, `lw`, `addi`, `subi`, `move`, `li`, `beq`, `jal` y `jr`.
- **Resultados:** Aplica correctamente la relación de recurrencia paridad$(n) = 1 -$ paridad$(n-1)$ a partir del caso base paridad$(0) = 0$, retornando 0 para enteros pares y 1 para impares.

6.2. Evaluación de la Función Iterativa

- **Gestión de la Pila (`$sp`):** No requiere el uso de la memoria pila, logrando una complejidad de almacenamiento constante $\mathcal{O}(1)$.
- **Instrucciones y Registros:** Emplea únicamente registros temporales: `$t0` como contador, `$t1` como acumulador y `$t2` como alternador de signo. Hace uso de `add`, `neg`, `subi`, `beq` y `j`.
- **Resultados:** Determina la paridad mediante la alternancia iterativa de signos sin sobrecargar los recursos de memoria.

6.3. Conclusiones Comparativas

- **Eficiencia Temporal:** La función iterativa ejecuta la tarea significativamente más rápido al omitir la latencia de acceso a memoria RAM y la sobrecarga de saltos de subrutina.
- **Seguridad de Memoria:** La solución iterativa es óptima y segura frente a valores de gran magnitud, eliminando por completo la posibilidad de fallos por desbordamiento de pila.